A. Advertising dataset — Regression

1. Comparative analysis of advertising channels

The Advertising dataset contains expenditure on TV, Radio, and Newspaper advertising and the corresponding sales. Develop regression models to identify which combination of advertising channels provides the best sales prediction.

Objectives:

1. Inspect and preprocess the dataset.
2. Train Linear Regression models using individual features, pairs of features, and all three features.
3. Compare RMSE and R^2 Score and identify the best-performing feature combination.

2. Linear versus polynomial regression

Investigate whether a nonlinear relationship between advertising expenditure and sales improves prediction accuracy.

Objectives:

1. Train a Linear Regression model using TV and Radio expenditure.
2. Apply Polynomial Features of degree 2 and train another regression model.
3. Compare the models using RMSE and R^2 Score on the same test set.

3. Evaluating the contribution of Newspaper advertising

Determine whether Newspaper expenditure provides useful information for predicting sales when TV and Radio expenditure are already available.

Objectives:

1. Train a multiple regression model using TV, Radio, and Newspaper.
2. Train another model after excluding Newspaper.
3. Compare RMSE, R^2 Score, and adjusted R^2 Score to assess the effect of removing the feature.

4. Identifying unusual sales predictions

Analyse the prediction errors generated by a regression model on the Advertising dataset.

Objectives:

1. Train a Multiple Linear Regression model using all advertising features.
2. Calculate the residual for each test observation and identify the five largest absolute residuals.
3. Plot actual versus predicted sales and discuss possible reasons for large prediction errors.

5. Effect of training data size on sales prediction

Investigate how the amount of training data influences regression performance.

Objectives:

1. Train regression models using 50%, 70%, and 80% of the dataset for training.
2. Use the same test set wherever applicable, with consistent preprocessing.
3. Compare RMSE and R^2 Score and discuss whether increasing training data improves prediction.

B. Used Cars dataset — Regression

6. Comparing regression algorithms for used-car prices

Predict used-car selling prices using year bought, kilometres driven, transmission, and owner.

Objectives:

1. Clean the data and encode categorical features.
2. Train Linear Regression, Decision Tree Regression, and Random Forest Regression models.
3. Compare RMSE and R^2 Score using the same train-test split and identify the best model.

7. Effect of vehicle age and kilometres driven

Examine how vehicle age and usage jointly influence the estimated selling price.

Objectives:

1. Derive vehicle age using a specified reference year and retain kilometres driven as a separate feature.
2. Train one regression model using kilometres driven alone and another using both features.
3. Compare RMSE and R^2 Score and analyse whether vehicle age improves prediction.

8. Hyperparameter tuning of a Decision Tree Regressor

Improve used-car price prediction by identifying suitable Decision Tree parameters.

Objectives:

1. Train a baseline Decision Tree Regressor.
2. Use GridSearchCV to tune `max_depth`, `min_samples_split`, and `min_samples_leaf`.
3. Compare the baseline and tuned models using cross-validation RMSE and test-set R^2 Score.

9. Impact of categorical encoding on price prediction

Investigate how the representation of transmission and owner affects regression performance.

Objectives:

1. Encode categorical features using an appropriate encoding technique.
2. Train a regression model using the encoded features alongside numerical attributes.
3. Compare the model with a numerical-features-only baseline using RMSE and R^2 Score.

10. Detecting and handling extreme car prices

Determine whether extreme selling-price observations influence the performance of a regression model.

Objectives:

1. Identify potential outliers in the selling-price column using the IQR method.
2. Train one model on the original training data and another after applying a documented outlier-handling method to the training data.
3. Compare both models on the same untouched test set using RMSE and R^2 Score.

11. Feature importance in used-car price prediction

Identify which available car attributes contribute most to predicting selling price.

Objectives:

1. Train a Random Forest Regressor using all available features.
2. Calculate and rank the feature importance scores.
3. Retrain a model using the three most important features and compare its RMSE and R^2 Score with the full-feature model.

C. Social Network Ads dataset — Classification

12. Comparing Linear and RBF Kernel SVM

Predict whether a user purchases a car using age and estimated salary.

Objectives:

1. Preprocess the dataset and standardise the numerical features.
2. Train Linear Kernel and RBF Kernel SVM classifiers using the same train-test split.
3. Compare confusion matrices, accuracy, precision, recall, and F1-score and identify the better-performing kernel.

13. Effect of feature scaling on SVM classification

Investigate whether feature scaling affects the prediction of car purchases.

Objectives:

1. Train an RBF Kernel SVM using unscaled age and estimated salary.
2. Train another RBF Kernel SVM using standardised features.
3. Compare their confusion matrices, accuracy, and F1-score and explain the observed differences.

14. SVM hyperparameter optimisation

Improve the performance of an SVM classifier by tuning its hyperparameters.

Objectives:

1. Standardise age and estimated salary.
2. Apply GridSearchCV to tune `C` and `gamma` for an RBF Kernel SVM.
3. Compare the default and tuned models using cross-validation scores, test accuracy, and F1-score.

15. Analysing the influence of age and salary

Determine whether age or estimated salary provides more useful information for predicting car purchases.

Objectives:

1. Train separate SVM classifiers using age alone, salary alone, and both features.
2. Use the same kernel, data split, and preprocessing approach for fair comparison.
3. Compare precision, recall, F1-score, and accuracy and identify the most effective feature combination.

16. Decision boundary visualisation for car purchases

Visualise how different SVM kernels separate customers who purchase a car from those who do not.

Objectives:

1. Standardise age and estimated salary.
2. Train Linear and RBF Kernel SVM classifiers and plot their decision boundaries.
3. Compare their decision regions and test-set performance.

17. Analysing classification errors

Investigate which customers are incorrectly classified by an SVM model.

Objectives:

1. Train an RBF Kernel SVM using the available customer features.

2. Identify false positives and false negatives from the test-set predictions.
3. Calculate precision and recall, and discuss the practical implications of the two types of errors.

18. Comparing SVM with Logistic Regression

Evaluate whether a nonlinear SVM provides better predictions than Logistic Regression for customer purchase behaviour.

Objectives:

1. Train Logistic Regression and RBF Kernel SVM classifiers using standardised features.
2. Use identical train-test partitions for both models.
3. Compare confusion matrices, accuracy, precision, recall, and F1-score and justify the preferred model.

D. Iris dataset — Classification and dimensionality reduction

19. Analysing Decision Tree depth and overfitting

Examine the effect of tree complexity on the classification of Iris species.

Objectives:

1. Train Decision Tree classifiers with maximum depths of 2, 3, 4, and 5.
2. Calculate training and testing accuracy for each model.
3. Plot the results and identify the depth that offers a suitable balance between underfitting and overfitting.

20. Comparing impurity criteria in Decision Trees

Investigate how the splitting criterion influences Iris classification.

Objectives:

1. Train Decision Tree classifiers using Gini and Entropy criteria with the same depth and split settings.
2. Compare their confusion matrices and classification accuracy.
3. Visualise the trees and discuss differences in their splitting decisions.

21. Comparing individual and ensemble classifiers

Determine whether ensemble methods improve classification performance compared with a single Decision Tree.

Objectives:

1. Train Decision Tree, Bagging, Random Forest, and AdaBoost classifiers.
2. Evaluate the models using the same stratified train-test split.
3. Compare accuracy, macro-precision, macro-recall, and macro-F1-score.

22. PCA with different numbers of principal components

Examine how dimensionality reduction affects Iris species classification.

Objectives:

1. Standardise the four input features and apply PCA.
2. Train the same SVM classifier using the original features, two principal components, and three principal components.
3. Compare accuracy, macro-F1-score, and cumulative explained variance to evaluate the trade-off between dimensionality and performance.

23. Comparing classifiers before and after PCA

Determine whether PCA changes the performance of different classification algorithms.

Objectives:

1. Train Logistic Regression and SVM classifiers on the standardised original Iris features.
2. Apply PCA to retain 95% of the training data's variance and retrain both classifiers on the transformed features.
3. Compare accuracy and macro-F1-score before and after PCA, ensuring PCA is fitted only on the training data.

24. Feature selection using Decision Tree importance

Investigate whether a reduced set of Iris measurements can achieve classification performance comparable to that of all four features.

Objectives:

1. Train a Decision Tree classifier and rank the four features by importance.
2. Retrain the classifier using the two most important features.
3. Compare accuracy, macro-F1-score, and confusion matrices for the full-feature and reduced-feature models.

25. Analysing misclassification between Iris species

Identify which Iris species are most frequently confused by a classifier and investigate whether a different model reduces these errors.

Objectives:

1. Train Linear Kernel SVM and RBF Kernel SVM classifiers using the same stratified data split and standardised features.

2. Generate confusion matrices and identify the most frequently misclassified species.
3. Compare per-class precision, recall, and F1-score and explain which classifier better distinguishes the species.

Implementation from scratch

26. Linear Regression from scratch using the Advertising dataset

Predict sales using TV advertising expenditure by implementing Linear Regression without Scikit-learn.

Objectives:

1. Load and preprocess the dataset using Pandas and NumPy.
2. Implement the least-squares method to calculate the slope and intercept without using `LinearRegression`.
3. Predict sales, calculate RMSE and \(R^2\), and plot the regression line with actual observations.

27. Multiple Linear Regression from scratch

Implement Multiple Linear Regression to predict sales using TV and Radio advertising expenditure.

Objectives:

1. Construct the feature matrix, including the intercept term.
2. Calculate the regression coefficients using the normal equation without Scikit-learn.
3. Predict sales and compare RMSE and \(R^2\) with a model using TV expenditure alone.

28. Gradient Descent for Linear Regression

Develop a Linear Regression model from scratch using gradient descent to predict sales from Radio advertising expenditure.

Objectives:

1. Initialise the slope and intercept and define the Mean Squared Error cost function.
2. Implement gradient descent with configurable learning rate and number of iterations.
3. Plot the cost against iterations and evaluate predictions using RMSE.

29. Logistic Regression from scratch using Social Network Ads

Predict whether a customer purchases a car using age and estimated salary by implementing Logistic Regression without Scikit-learn.

Objectives:

1. Preprocess the dataset and standardise numerical features using manually implemented calculations.
2. Implement the sigmoid function, binary cross-entropy loss, and gradient descent to learn the model parameters.
3. Generate binary predictions using a threshold of 0.5 and evaluate the model using a confusion matrix, accuracy, precision, and recall.

30. Effect of learning rate on Logistic Regression

Investigate how the learning rate influences the convergence and classification performance of a Logistic Regression model implemented from scratch.

Objectives:

1. Implement Logistic Regression using the Social Network Ads dataset without Scikit-learn.
2. Train separate models using learning rates of 0.001, 0.01, and 0.1 while keeping other settings fixed.
3. Plot the loss curves and compare convergence behaviour, accuracy, and F1-score.

31. Multiclass Logistic Regression from scratch using Iris

Implement a multiclass Logistic Regression classifier to predict Iris species without using Scikit-learn's classification algorithms.

Objectives:

1. Preprocess the Iris dataset and encode the three target classes using one-hot encoding.
2. Implement the softmax function, categorical cross-entropy loss, and gradient descent for multiclass classification.
3. Predict the Iris species and evaluate the model using a confusion matrix, accuracy, and macro-F1-score.